TFLite INT8 Export of a Keyword Spotting Transformer via AWQ Quantization

A self-contained technical reference for converting PyTorch transformer models to TFLite

---

Zentree Labs

Keyword Spotting Project

Toolchain:	PyTorch 2.3 → litert-torch → ai_edge_quantizer → TFLite
Target:	Raspberry Pi 3 (armv7l, Python 3.9, tflite-runtime 2.13)
Quantization:	AWQ (Activation-aware Weight Quantization) + INT8 FULLY_CONNECTED
Dataset:	Google Speech Commands v2 (35 keywords)

This document is intended to be self-contained. A reader should be able to reproduce the full pipeline and understand every binary-level detail of the output file without consulting any external reference.

Contents

1 Introduction and Motivation **2**

2 Model Architecture **2**
2.1 Overview . . . 2
2.2 Hyperparameters . . . 3
2.3 CLS Token Design . . . 3
2.4 Forward Pass . . . 3
2.5 Causal Self-Attention . . . 3
2.6 Feed-Forward Network . . . 4

3 Audio Preprocessing **4**
3.1 Pipeline . . . 4
3.2 STFT Parameters . . . 4
3.3 Inference Fallback . . . 5

4 Training **5**
4.1 Dataset . . . 5
4.2 Training Configuration . . . 5
4.3 Checkpoint . . . 5

5 AWQ: Theory and Implementation **6**
5.1 Background: Why Standard INT8 Fails for Transformers . . . 6
5.2 AWQ Algorithm (Lin et al., 2023) . . . 6
5.3 Pseudo-Quantization Function . . . 7
5.4 Step 1: Activation Calibration . . . 7
5.5 Step 2: AWQ Scale Grid Search . . . 7
5.6 Step 3: Folding Scales into LayerNorm Parameters . . . 8

6 TFLite Export Pipeline **9**
6.1 Toolchain Overview . . . 9
6.2 Stage 1: litert-torch (PT2E Tracing) . . . 9
6.3 Stage 2: ai_edge_quantizer (INT8 Quantization) . . . 9

7 The TFLite FlatBuffers Binary Format **10**
7.1 FlatBuffers Fundamentals . . . 10
7.2 TFLite Schema (Relevant Subset) . . . 11
7.3 Navigating from File Root to `Model.buffers` . . . 12
7.4 Two Buffer Formats . . . 12
7.4.1 Classic Embedded Format (pre-LiteRT) . . . 12
7.4.2 LiteRT mmap Format (ai_edge_quantizer output) . . . 13
7.5 Distinguishing the Two Formats . . . 13

8 Compatibility Patches **13**
8.1 Patch 1: Hidden Inputs (`_fix_hidden_inputs`) . . . 13
8.2 Patch 2: Buffer Embedding (`_embed_mmap_buffers`) . . . 14
8.2.1 Building a new Buffer table in FlatBuffers binary . . . 14
8.2.2 Updating the slot . . . 14
8.2.3 Full implementation . . . 14

8.2.4 File size after patching . . . 16

9 End-to-End Export Workflow **16**
9.1 Running the Pipeline . . . 16
9.2 Copying to Raspberry Pi . . . 17
9.3 Preprocessing Config . . . 17

10 Inference on Raspberry Pi 3 **17**
10.1 Installation . . . 17
10.2 Model Loading and Hidden-Input Patching . . . 17
10.3 Inference Modes . . . 18
10.4 Inference Latency . . . 18

11 Adapting the Pipeline to a Different Model **18**
11.1 Prerequisites . . . 18
11.2 AWQ Targeting . . . 19
11.3 Calibration Data . . . 19
11.4 Quantization Configuration . . . 19
11.5 Detecting and Handling the mmap Format . . . 19
11.6 Verifying the Output . . . 20

12 Troubleshooting **20**

13 Summary of Output Files **21**

A FlatBuffers Navigation Reference **21**

B Vtable Field Index Quick Reference **22**

1 Introduction and Motivation

This report documents the complete pipeline for converting a decoder-only Transformer trained in PyTorch to a TFLite INT8 file suitable for inference on a Raspberry Pi 3 (armv7l, 1 GB RAM, quad-core Cortex-A53 at 1.2 GHz).

The pipeline has four conceptually distinct stages:

1. **Model training.** A `KeywordSpottingTransformer` is trained in FP32 on Google Speech Commands v2 using PyTorch.
2. **AWQ weight transformation.** The FP32 model weights are rescaled using Activation-aware Weight Quantization (AWQ) so that subsequent INT8 quantization loses as little accuracy as possible. This step is *mathematically lossless*: the transformed model produces identical outputs (to floating-point precision) as the original.
3. **TFLite INT8 export.** The AWQ-transformed model is exported to a TFLite flatbuffer via `litert-torch` (PyTorch 2 Export / PT2E) and then statically quantized to INT8 by `ai_edge_quantizer`.
4. **Compatibility patching.** The output of `ai_edge_quantizer` uses the LiteRT *mmap format* (weights stored at raw byte offsets), which old `tflite-runtime` versions do not support. The file is patched at the FlatBuffers binary level to embed all weight data inside `Buffer.data` vectors—the format every version of `tflite-runtime` understands.

The rest of this document covers each stage in enough detail that a reader can implement it from scratch.

2 Model Architecture

2.1 Overview

The model is a *decoder-only* Transformer—identical in structure to GPT—applied to sequence classification. Its input is a log-mel spectrogram of shape $[T, F]$ where $T = 101$ frames (approximately one second at 10 ms hop) and $F = 80$ mel bins. Its output is a vector of 35 logits, one per keyword.

2.2 Hyperparameters

Parameter	Symbol	Value
Embedding dimension	d_{model}	256
Attention heads	H	4
Decoder layers	L	4
FFN hidden size	d_{ff}	512
Dropout	p	0.1 (training only)
Max sequence length	$T_{\max}$	101 (+ 1 CLS token)
Number of classes	C	35
Total parameters		≈6.7 M

Table 1: Model hyperparameters

2.3 CLS Token Design

Rather than prepending the CLS token (as in BERT), it is *appended* at position $T + 1$. Because attention is *causal* (upper-triangular mask), the last position attends to all preceding frames without any future-leakage concern. This makes it a natural pooling point: only the CLS token's representation is passed to the classification head.

Why causal attention for classification? In keyword spotting the model sees the entire 1-second clip, so non-causal attention would also be valid. The causal design was chosen for forward compatibility: the same architecture and weights could be fine-tuned for streaming (frame-by-frame) inference without architectural changes.

2.4 Forward Pass

Let $\mathbf{X} \in \mathbb{R}^{B\times T\times F}$ be a batch of mel spectrograms.

1. **Input projection:** $\mathbf{H} = \mathbf{X}\,\mathbf{W}_{\text{in}} + \mathbf{b}_{\text{in}}$, $\mathbf{H} \in \mathbb{R}^{B\times T\times d}$.
2. **CLS append:** concatenate learnable $\mathbf{c} \in \mathbb{R}^{1\times 1\times d}$ to give $\tilde{\mathbf{H}} \in \mathbb{R}^{B\times (T+1)\times d}$.
3. **Positional embedding:** add $\text{Embed}(0, \ldots, T)$, a learned table of shape $[(T_{\max} + 1) \times d]$.
4. **Decoder blocks ($\times L$):**

$$\mathbf{H} \leftarrow \mathbf{H} + \text{CausalMHA}(\text{LN}_1(\mathbf{H})) \tag{1}$$
$$\mathbf{H} \leftarrow \mathbf{H} + \text{FFN}(\text{LN}_2(\mathbf{H})) \tag{2}$$

5. **Output LN + head:** $\mathbf{y} = \text{LN}(\tilde{\mathbf{H}})_{:,-1,:}\,\mathbf{W}_{\text{head}}$. Only the last (CLS) position is read.

2.5 Causal Self-Attention

For a single head with dimension $d_h = d/H$:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}} + \mathbf{M}\right)\mathbf{V} \tag{3}$$

where $\mathbf{M}_{ij} = -10^4$ if $j > i$ else 0 (causal mask). Queries, keys, and values are produced by a single fused linear projection $\mathrm{QKV} : \mathbb{R}^d \to \mathbb{R}^{3d}$, then split and reshaped.

2.6 Feed-Forward Network

$$\mathrm{FFN}(\mathbf{x}) = \mathbf{W}_2\,\mathrm{GELU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 \tag{4}$$

The two linear layers (`net[0]`, `net[3]`) in the FFN are the primary quantization targets for AWQ because they operate on LayerNorm-normalized activations, which may have large per-channel variance.

3 Audio Preprocessing

3.1 Pipeline

Raw waveform → pad/truncate to SR = 16000 samples → short-time Fourier transform → mel filterbank → log → instance normalization.

3.2 STFT Parameters

Parameter	Value	Note
Sample rate	16 000 Hz	
FFT size (N_{FFT})	512	frequency resolution
Window length	400 samples	25 ms
Hop length	160 samples	10 ms, 75% overlap
Mel bins (F)	80	
Max frames (T)	101	covers 1 second

Table 2: STFT / mel spectrogram parameters

The power spectrogram at frame t, bin k is:

$$S_{t,k} = \Big|\mathrm{STFT}(x)[t,k]\Big|^2 \tag{5}$$

Mel filterbank matrix $\mathbf{F} \in \mathbb{R}^{80\times 257}$ maps the 257 FFT bins to 80 triangular mel-spaced filters (HTK formula):

$$m = 2595\log_{10}\left(1 + \frac{f}{700}\right) \tag{6}$$

Log-mel spectrogram: $\mathbf{M}_{t,n} = \log\Big(\mathbf{F}_{n,:}\,\mathbf{S}_{:,t} + \epsilon\Big)$, $\epsilon = 10^{-6}$.

Instance normalization (per-clip):

$$\hat{\mathbf{M}} = \frac{\mathbf{M} - \mu_{\mathbf{M}}}{\sigma_{\mathbf{M}} + 10^{-5}} \tag{7}$$

3.3 Inference Fallback

On the Raspberry Pi where `librosa` may be unavailable, `inference_rpi_awq.py` implements a pure NumPy STFT and mel filterbank (`_stft_numpy` + `_mel_filterbank`). The filterbank construction follows the HTK triangular design:

1. Convert $[0, f_s/2]$ to mel scale with $F + 2$ equally spaced points.
2. Convert back to Hz; map to FFT bin indices.
3. Fill each row m with a rising ramp from bin b_{m-1} to b_m and a falling ramp from b_m to b_{m+1}.

4 Training

4.1 Dataset

Google Speech Commands v2 contains ∼105 000 one-second audio clips across 35 keyword classes plus silence/background. This project uses only the 35 keyword classes (no silence class), giving approximately:

- Training: ∼84 000 examples
- Validation: ∼9 900 examples

4.2 Training Configuration

Setting	Value
Optimizer	AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.98$
Learning rate schedule	OneCycleLR, $\mathrm{lr}_{\max} = 3 \times 10^{-4}$
Weight decay	10^{-4}
Batch size	256
Epochs	40
Loss	Cross-entropy with label smoothing $\epsilon = 0.1$
Gradient clip	$\lVert\cdot\rVert_2 \leq 1.0$
Data augmentation	SpecAugment: `FrequencyMasking(10)`, `TimeMasking(20)`

Table 3: Training hyperparameters

4.3 Checkpoint

The best-validation-accuracy checkpoint is saved to `checkpoints/best_model.pt` as a dictionary:

```
{
    "epoch":        int,
    "model_state": OrderedDict,    # nn.Module.state_dict()
    "val_acc":      float,
    "config":       dict,          # constructor kwargs for
   reproducibility
```

```
}
```

5 AWQ: Theory and Implementation

5.1 Background: Why Standard INT8 Fails for Transformers

In symmetric per-output-channel INT8 quantization, each weight row $\mathbf{w}_i$ is quantized as:

$$\hat{\mathbf{w}}_i = \text{round}\left(\frac{\mathbf{w}_i}{s_i}\right) \cdot s_i, \quad s_i = \frac{\max_j |w_{ij}|}{127} \tag{8}$$

For a fully-connected layer $\mathbf{y} = \mathbf{W}\mathbf{x}$, the per-output-channel scale s_i protects large weights but ignores the *activation* $\mathbf{x}$. If input channel j has a large activation x_j, then the quantization error δw_{ij} is amplified by a factor of x_j in the output:

$$\delta y_i = \sum_j \delta w_{ij} \cdot x_j \tag{9}$$

Channels with large x_j (the *salient channels*) dominate the output error, yet standard quantization treats all input channels equally.

5.2 AWQ Algorithm (Lin et al., 2023)

AWQ protects salient channels by applying a per-input-channel scale $\mathbf{s}$ before quantization:

$$\mathbf{W}\mathbf{x} = \underbrace{\frac{\mathbf{W}}{\mathbf{s}^\top}}_{\tilde{\mathbf{W}}} \cdot \underbrace{(\mathbf{s} \odot \mathbf{x})}_{\tilde{\mathbf{x}}} \tag{10}$$

Scaling up $\tilde{\mathbf{x}}_j = s_j x_j$ squeezes the dynamic range of $\tilde{\mathbf{W}}_{:,j} = \mathbf{W}_{:,j}/s_j$, reducing its quantization error. The scale s_j is chosen proportional to x_j^α for some $\alpha \in [0, 1]$, then normalized to preserve the overall magnitude:

$$s_j = \frac{x_{j,\max}^\alpha}{\text{mean}(\mathbf{x}_{\max}^\alpha)}, \quad \mathbf{x}_{\max} = \max_{\text{calibration}} |x_j| \tag{11}$$

The optimal α is found by a grid search that minimizes the mean-squared quantization error of the scaled weights:

$$\alpha^* = \arg\min_\alpha \left\| \mathbf{W} - \hat{\tilde{\mathbf{W}}}(\alpha) \odot \mathbf{s}(\alpha)^\top \right\|_F^2 \tag{12}$$

where $\hat{\tilde{\mathbf{W}}}(\alpha)$ is the pseudo-quantized version of $\mathbf{W}/\mathbf{s}(\alpha)^\top$.

Intuition.

- $\alpha = 0$: $\mathbf{s} = \mathbf{1}$, no change (standard quantization).
- $\alpha = 1$: $\mathbf{s} = \mathbf{x}_{\max}$ (full SmoothQuant-style equalization).
- Optimum is typically $\alpha \approx 0.5$, varies per layer.

5.3 Pseudo-Quantization Function

Symmetric per-output-channel INT8:

$$\mathrm{Q}(\mathbf{W}) = \mathrm{clip}\left(\mathrm{round}\left(\frac{\mathbf{W}}{s}\right), -128, 127\right) \cdot s, \quad s_i = \frac{\max_j |W_{ij}|}{127} \tag{13}$$

This simulates the rounding error that will be introduced at inference time without actually converting to integer dtype.

```
def _pseudo_quantize(w: torch.Tensor, n_bits: int) -> torch.Tensor:
    q_max  = 2 ** (n_bits - 1) - 1            # 127 for n_bits=8
    scale  = w.abs().max(dim=1, keepdim=True).values.clamp(min=1e-8) / q_max
    w_int  = (w / scale).round().clamp(-q_max - 1, q_max)
    return w_int * scale
```

5.4 Step 1: Activation Calibration

An `ActivationCalibrator` registers forward hooks on every `nn.Linear` module. For each calibration batch the hook records the per-input-channel maximum absolute activation:

```
def hook(_module, inputs, _output):
    x = inputs[0].detach().float()
    ch_max = x.reshape(-1, x.shape[-1]).abs().max(dim=0).values
    # running maximum across all calibration batches
    if name in self.scales:
        self.scales[name] = torch.maximum(self.scales[name], ch_max)
    else:
        self.scales[name] = ch_max.clone()
```

The reshape `(-1, x.shape[-1])` flattens the batch and time dimensions so that `.max(dim=0)` gives the maximum over all tokens across all sequences, yielding a d_{in}-dimensional vector of activation maxima.

64 calibration batches of 32 samples each (2048 total) are used.

5.5 Step 2: AWQ Scale Grid Search

For each target Linear layer, `_find_awq_scale` searches $\alpha \in \{0/N, 1/N, \ldots, N/N\}$ with $N = 20$:

```
for step in range(n_grid + 1):        # 21 values in [0, 1]
    alpha   = step / n_grid
    scale   = (act_max ** alpha).clamp(min=1e-8)
    scale   = scale / scale.mean()    # normalise

    w_scaled = w / scale.unsqueeze(0)              # [out, in]
```

```
    w_back   = _pseudo_quantize(w_scaled, n_bits) * scale.unsqueeze(0)
    err      = (w - w_back).pow(2).mean().item()

    if err < best_err:
        best_err, best_alpha = err, alpha
```

Targeted layers per decoder block:

- `blocks.i.attn.qkv` — the QKV projection (receives LayerNorm output, size $[d_{\text{model}}, 3d_{\text{model}}]$)
- `blocks.i.ff.net.0` — the first FFN linear (receives LayerNorm output, size $[d_{\text{model}}, d_{\text{ff}}]$)

5.6 Step 3: Folding Scales into LayerNorm Parameters

The AWQ scale vector $\mathbf{s}$ must be *absorbed* into the model so that the graph structure remains unchanged. Because every targeted Linear is preceded by a LayerNorm, we fold $\mathbf{s}$ into the LayerNorm's gain (γ) and bias (β):

Theorem 5.1 (AWQ Folding Equivalence)**.** *Let* $\mathbf{s} \in \mathbb{R}^d_{>0}$*. Define the transformed parameters:*

$$\tilde{\gamma}_j = s_j \cdot \gamma_j \tag{14}$$

$$\tilde{\beta}_j = s_j \cdot \beta_j \tag{15}$$

$$\tilde{\mathbf{W}}_{:,j} = \mathbf{W}_{:,j}/s_j \tag{16}$$

Then for any normalized input $\hat{\mathbf{x}}$*:*

$$\tilde{\mathbf{W}}\,(\tilde{\gamma} \odot \hat{\mathbf{x}} + \tilde{\beta}) = \mathbf{W}\,(\gamma \odot \hat{\mathbf{x}} + \beta) \tag{17}$$

Proof.

$$\tilde{\mathbf{W}}\,(\tilde{\gamma} \odot \hat{\mathbf{x}} + \tilde{\beta}) = \frac{\mathbf{W}}{\mathbf{s}^\top} \cdot (\mathbf{s} \odot \gamma \odot \hat{\mathbf{x}} + \mathbf{s} \odot \beta) \tag{18}$$

$$= \frac{\mathbf{W}}{\mathbf{s}^\top} \cdot \mathbf{s}^\top \odot (\gamma \odot \hat{\mathbf{x}} + \beta) \tag{19}$$

$$= \mathbf{W}\,(\gamma \odot \hat{\mathbf{x}} + \beta) \qquad \square \tag{20}$$

□

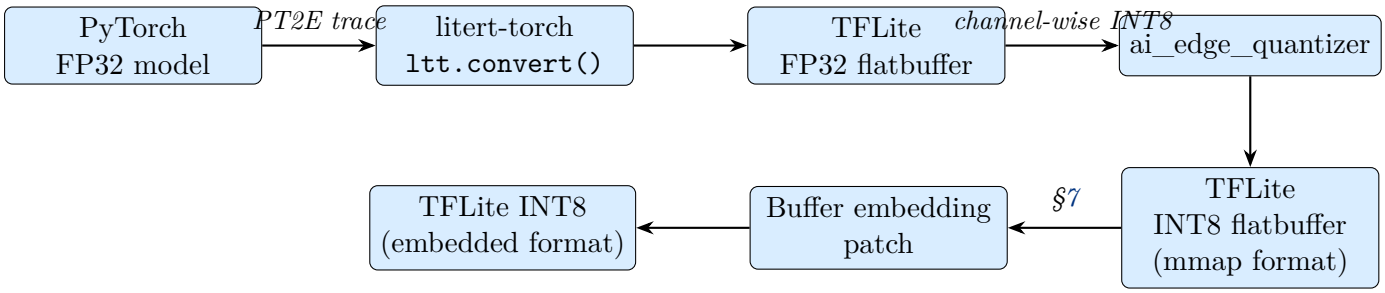
In code (operating on `model.blocks[i]`):

```
linear.weight.data.div_(s.unsqueeze(0))    # W[:, j] /= s[j]
norm.weight.data.mul_(s)                   # gamma[j] *= s[j]
if norm.bias is not None:
    norm.bias.data.mul_(s)                 # beta[j] *= s[j]
```

After folding, the model is verified to be mathematically equivalent to the original (assertion: accuracy difference $< 0.5\%$ on 20 validation batches).

6 TFLite Export Pipeline

6.1 Toolchain Overview



6.2 Stage 1: litert-torch (PT2E Tracing)

`litert-torch` converts a PyTorch `nn.Module` to a TFLite flatbuffer using PyTorch 2 Export (PT2E):

```
import litert_torch as ltt
fp32_bytes: bytes = ltt.convert(model, sample_inputs).
    model_content()
```

Where `sample_inputs` is a tuple of representative input tensors (shape and dtype must match production inputs):

```
sample = (torch.zeros(1, MAX_FRAMES, N_MELS),)  # [1, 101, 80]
```

Internally, PT2E:

1. Runs `torch.export.export(model, sample)` to produce a computation graph via *symbolic tracing* with no Python control flow.
2. Lowers the graph to TFLite operators (primarily `FULLY_CONNECTED` for all linear projections and `SOFTMAX` for attention).
3. Lifts model parameters (weights and biases) as additional graph *inputs* (this is a side effect of PT2E's functional representation).
4. Serializes to FlatBuffers TFLite format.

PT2E parameter lifting. PT2E treats model parameters as tensor inputs to the exported graph. `ai_edge_quantizer` then quantizes them to INT8 constants—but may leave them in the `SubGraph.inputs` list in some versions. This is checked and corrected by `_fix_hidden_inputs()` (see Section 8.1).

6.3 Stage 2: ai_edge_quantizer (INT8 Quantization)

```
from ai_edge_quantizer import quantizer as aq

qt = aq.Quantizer(bytearray(fp32_bytes))
qt.add_dynamic_config(
    regex=".*",
```

```
    operation_name=aq._TFLOpName.FULLY_CONNECTED,
    num_bits=8,
    granularity=aq.qtyping.QuantGranularity.CHANNELWISE,
)
int8_bytes = bytes(qt.quantize().quantized_model)
```

The call to `add_dynamic_config` configures *channel-wise INT8* post-training quantization for every `FULLY_CONNECTED` operator (the TFLite lowering of `nn.Linear`).

What ai_edge_quantizer does.

1. Parses the input FP32 TFLite flatbuffer.
2. For each targeted operator, replaces the FP32 weight tensor with an INT8 tensor and records per-channel scale/zero-point metadata in the tensor's `quantization` field.
3. Inserts quantize/dequantize operator nodes around the INT8 weights where necessary.
4. Re-serializes to a new TFLite flatbuffer.

Output format: LiteRT mmap. The re-serialized flatbuffer uses the *LiteRT mmap format*: each `Buffer` table has its weight data stored at a raw byte offset within the *file* rather than in a FlatBuffers `[ubyte]` vector. This is described in detail in Section 7.

7 The TFLite FlatBuffers Binary Format

Understanding the binary format is essential for implementing the compatibility patch in Section 8.

7.1 FlatBuffers Fundamentals

FlatBuffers is a binary serialization format designed for zero-copy access. Unlike Protocol Buffers, a FlatBuffer can be accessed directly from memory without deserialization.

Tables. A FlatBuffers *table* is the basic composite type. It is stored as two contiguous regions:

1. **Vtable** (variable-length, stored *before* the object):

`vtable_size`	`object_size`	`field[0]_offset`	`field[1]_offset`	`...`
u16 (2B)	u16 (2B)	u16 (2B)	u16 (2B)	

   Each `field[N]_offset` is the byte offset of that field *within the object* (0 if the field is absent/default).

2. **Object** (fixed-size):

`soffset`	`field[0]`	`field[1] ...`
i32 (4B)	field payloads	

   The first 4 bytes of every table object is the `soffset` (*signed* offset to the vtable).

Critical sign convention. The vtable address is computed as:

$$\text{vtable_addr} = \text{object_addr} - \text{soffset_value} \tag{21}$$

Because the vtable is stored *before* the object, the soffset value is *positive* (object > vtable). A common implementation mistake is to add instead of subtract, navigating to a garbage address.

Vectors. A FlatBuffers vector of type T is laid out as:

$$[\underbrace{N}_{\text{count, u32}}][\underbrace{e_0}_{T}][\underbrace{e_1}_{T}]\cdots[\underbrace{e_{N-1}}_{T}] \tag{22}$$

A field that points to a vector stores a u32 *relative* offset from the field's own byte address to the vector's `count` element:

$$\text{vector_addr} = \text{field_addr} + \text{field_value} \tag{23}$$

Object references. Within a vector of tables (e.g., `Model.buffers`), each slot stores a u32 relative offset from the *slot's address* to the table object:

$$\text{table_addr} = \text{slot_addr} + \text{slot_value} \tag{24}$$

7.2 TFLite Schema (Relevant Subset)

```
table Model {
  version:        uint;           // field 0
  operator_codes: [OperatorCode]; // field 1
  subgraphs:      [SubGraph];     // field 2
  description:    string;         // field 3
  buffers:        [Buffer];       // field 4
  metadata_buffer:[int];          // field 5
  metadata:       [Metadata];     // field 6
  signature_defs: [SignatureDef]; // field 7
}

table SubGraph {
  tensors: [Tensor];   // field 0
  inputs:  [int];      // field 1
  outputs: [int];      // field 2
  operators:[Operator];// field 3
  name:    string;     // field 4
}

table Buffer {
  data:   [ubyte] (force_align: 16); // field 0     classic format
  offset: int64;                     // field 1     mmap format
  size:   uint64;                    // field 2     mmap format
}
```

7.3 Navigating from File Root to `Model.buffers`

```python
import struct

def navigate(buf: bytes):
    u32 = lambda off: struct.unpack_from("<I", buf, off)[0]
    i32 = lambda off: struct.unpack_from("<i", buf, off)[0]
    u16 = lambda off: struct.unpack_from("<H", buf, off)[0]

    root = u32(0)                    # absolute offset to Model object
    vtbl = root - i32(root)          # vtable = obj - soffset (Eq. 1)

    # Model.buffers = field 4 -> vtable entry at vtbl + 4 + 4*2 = vtbl+12
    bufs_off = u16(vtbl + 12)        # offset of buffers field within Model obj
    bufs_ptr = root + bufs_off       # address of the field slot
    bufs_vec = bufs_ptr + u32(bufs_ptr)  # address of the buffers vector
    bufs_cnt = u32(bufs_vec)         # number of Buffer objects
```

The vtable entry for field N is at address $\text{vtbl} + 4 + 2N$ (4 bytes for the vtable-size and object-size fields, then 2 bytes per field).

7.4 Two Buffer Formats

7.4.1 Classic Embedded Format (pre-LiteRT)

`Buffer.data` (field 0) contains a FlatBuffers `[ubyte]` vector. The field slot stores a u32 relative offset to the vector's count element. The weight bytes immediately follow the count.

```
Buffer object layout:
  [soffset: i32] [data_field: u32 -> vector]

Vector layout (force_align:16 means first byte at 16-aligned address):
  [count: u32] [byte_0] [byte_1] ... [byte_{count-1}]
```

Reading:

```python
bobj     = slot + u32(slot)          # Buffer object start (Eq. 2)
bvtbl    = bobj - i32(bobj)          # Buffer vtable
data_off = u16(bvtbl + 4)            # field 0 at vtable[4]
data_ptr = bobj + data_off
data_vec = data_ptr + u32(data_ptr)
count    = u32(data_vec)
data     = buf[data_vec + 4 : data_vec + 4 + count]
```

7.4.2 LiteRT mmap Format (ai_edge_quantizer output)

`Buffer.offset` (field 1, i64) and `Buffer.size` (field 2, u64) store the byte offset and length within the TFLite *file* itself. `Buffer.data` (field 0) is absent (the vtable entry is 0).

Reading:

```
bobj      = slot + u32(slot)
bvtbl     = bobj - i32(bobj)
bvtsz     = u16(bvtbl)              # total vtable size in bytes
off_off   = u16(bvtbl + 6)          # field 1 at vtable[6]
size_off  = u16(bvtbl + 8)          # field 2 at vtable[8]
file_off  = struct.unpack_from("<q", buf, bobj + off_off)[0]  # signed
file_size = struct.unpack_from("<Q", buf, bobj + size_off)[0]  # unsigned
data      = buf[file_off : file_off + file_size]
```

Compatibility. `tflite-runtime` $\geq$ 2.14 (LiteRT) reads both formats. `tflite-runtime` 2.13 and earlier (the last version with armv7l wheels for Python 3.9) reads *only* the classic embedded format. On RPi 3, the mmap format causes every weight buffer to appear empty, and old runtimes then report: `RuntimeError: Input tensor N lacks data`.

7.5 Distinguishing the Two Formats

Check the vtable of a Buffer object for the presence/absence of each field:

```
bvtsz     = u16(bvtbl)
data_voff = u16(bvtbl + 4) if bvtsz >= 6  else 0  # 0 = absent
off_voff  = u16(bvtbl + 6) if bvtsz >= 8  else 0
size_voff = u16(bvtbl + 8) if bvtsz >= 10 else 0

is_mmap   = (not data_voff) and bool(off_voff) and bool(size_voff)
is_embedded = bool(data_voff)
is_empty  = (not data_voff) and (not off_voff)   # Buffer[0] is always empty
```

8 Compatibility Patches

8.1 Patch 1: Hidden Inputs (`_fix_hidden_inputs`)

In some versions of PT2E and `ai_edge_quantizer`, weight tensors that were graph inputs (due to PT2E parameter lifting) remain listed in `SubGraph.inputs` even after quantization. `tf.lite.Interpreter` auto-initialises them; old `tflite-runtime` does not and raises `Input tensor N lacks data`.

The fix compares the flatbuffer's `SubGraph.inputs` vector with the inputs reported by `tf.lite.Interpreter` (which only lists *true* inputs) and removes the extra entries.

Current model status. Inspection of the exported model confirms that `SubGraph.inputs = [0]` (only the mel spectrogram tensor), so this patch is a no-op for the current model but remains in place for robustness.

8.2 Patch 2: Buffer Embedding (`_embed_mmap_buffers`)

This is the primary compatibility fix. For every `Buffer` object in the mmap format, a new Buffer table with the bytes embedded in `Buffer.data` is appended to the file, and the slot in the `Model.buffers` vector is updated to point to it.

8.2.1 Building a new Buffer table in FlatBuffers binary

A new `Buffer` table with only field 0 (`data`) present has the following layout (all little-endian):

Offset	Size	Type	Value	Meaning
P	2 B	u16	6	vtable_size (3 fields × 2 B)
$P+2$	2 B	u16	8	object_size (soffset + data_ptr)
$P+4$	2 B	u16	4	field 0 offset within object
$P+6$	2 B	—	0, 0	alignment padding
$P+8$	4 B	i32	8	soffset: $(P+8) - P = 8$
$P+12$	4 B	u32	4	data field: vector is 4 bytes ahead
$P+16$	4 B	u32	N	vector count (= data size in bytes)
$P+20$	N B	u8[]	...	weight data bytes

P is the absolute file offset where the vtable starts. Alignment constraint: $P+20$ must be divisible by 16 (`force_align:16` for `Buffer.data`). Equivalently:

$$(P+20) \bmod 16 = 0 \quad \Leftrightarrow \quad P \bmod 16 = 12 \tag{25}$$

To satisfy (25), zero-padding bytes are appended before the vtable until the condition holds. Since $20 \bmod 4 = 0$, this alignment also guarantees the table object at $P+8$ is 4-byte aligned.

8.2.2 Updating the slot

After appending the new table at absolute position $P+8$, the slot in the buffers vector (at `slot_pos`) is updated using Equation 24:

$$\text{slot_value} \leftarrow (P+8) - \text{slot_pos} \tag{26}$$

The old mmap `Buffer` objects become unreferenced dead bytes in the file. No other part of the flatbuffer is modified.

8.2.3 Full implementation

```python
def _embed_mmap_buffers(tflite_bytes: bytes) -> bytes:
    orig = bytearray(tflite_bytes)

    def u32(o): return struct.unpack_from("<I", orig, o)[0]
```

```
    def i32(o): return struct.unpack_from("<i", orig, o)[0]
    def u16(o): return struct.unpack_from("<H", orig, o)[0]
    def i64(o): return struct.unpack_from("<q", orig, o)[0]
    def u64(o): return struct.unpack_from("<Q", orig, o)[0]

    root     = u32(0)
    vtbl     = root - i32(root)          # correct sign: obj - soffset
    bufs_off = u16(vtbl + 12)            # Model.buffers = field 4
    bufs_ptr = root + bufs_off
    bufs_vec = bufs_ptr + u32(bufs_ptr)
    bufs_cnt = u32(bufs_vec)

    extra = bytearray()
    for bidx in range(bufs_cnt):
        slot_pos = bufs_vec + 4 + bidx * 4
        bobj     = slot_pos + u32(slot_pos)
        bvtbl    = bobj - i32(bobj)
        bvtsz    = u16(bvtbl)

        data_voff = u16(bvtbl + 4) if bvtsz >= 6  else 0
        off_voff  = u16(bvtbl + 6) if bvtsz >= 8  else 0
        size_voff = u16(bvtbl + 8) if bvtsz >= 10 else 0

        if data_voff or not off_voff or not size_voff:
            continue   # skip embedded, skip empty

        file_off  = i64(bobj + off_voff)
        file_sz   = u64(bobj + size_voff)
        if file_sz == 0:
            continue

        data_bytes = bytes(orig[file_off : file_off + file_sz])

        # Satisfy force_align:16: (P+20) % 16 == 0
        while (len(orig) + len(extra) + 20) % 16 != 0:
            extra.append(0)
        P = len(orig) + len(extra)

        # Vtable (6 B) + pad (2 B) + table (8 B) + vector (4 + N B)
        extra.extend(struct.pack("<HHH", 6, 8, 4))     # vtable
        extra.extend(b"\x00\x00")                       # pad
        extra.extend(struct.pack("<i", 8))              # soffset
        extra.extend(struct.pack("<I", 4))              # data -> vector
        extra.extend(struct.pack("<I", len(data_bytes)))
        extra.extend(data_bytes)

```

```
        table_pos = P + 8
        struct.pack_into("<I", orig, slot_pos, table_pos - slot_pos)

    return bytes(orig) + bytes(extra)
```

8.2.4 File size after patching

The original file contains both the FlatBuffers metadata section (∼150 kB) and the mmap weight data (∼2.28 MB), for a total of ∼2.4 MB. After embedding, the same weight bytes appear a second time in the `Buffer.data` vectors appended at the end. The patched file is therefore approximately 2× the original size (∼4.8 MB). This increase is acceptable for RPi 3 (1 GB RAM).

9 End-to-End Export Workflow

9.1 Running the Pipeline

On the training machine (Linux/Windows/macOS, GPU optional):

```
# 1. Install dependencies
pip install -r requirements_training.txt

# 2. Train (downloads ~2.4 GB dataset on first run)
python train.py

# 3. AWQ quantization + TFLite export
python quantize_awq.py
```

Expected output of `quantize_awq.py`:

```
Loading checkpoint...
  Loaded  (training val_acc: 0.9XXX)

[1/4] Calibrating activation importance (64 batches)...
  Collected scales for 32 Linear layers

[2/4] Searching AWQ scales (grid=20, bits=8)...
  AWQ grid-search: blocks.0.attn.qkv ... done
  AWQ grid-search: blocks.0.ff.net.0 ... done
  ... (8 layers total)

[3/4] Applying AWQ weight transformations...
  FP32 accuracy  original=0.9XXX  after AWQ=0.9XXX

[4/4] Exporting to TFLite INT8 (PT2E + litert-torch)...
  No hidden inputs found -- model is already clean.
  Converted 65/66 buffers: mmap (offset/size) -> embedded (Buffer.data
   )
  Saved: checkpoints/keyword_spotting_awq.tflite  (4.78 MB)

Validating (30 batches each)...
  FP32  (AWQ) : 0.9XXX
  INT8 TFLite : 0.9XXX
```

```
  Accuracy drop: 0.00XX
```

9.2 Copying to Raspberry Pi

```
scp checkpoints/keyword_spotting_awq.tflite  pi@<pi-ip>:~/keyword/
    checkpoints/
scp checkpoints/preprocess_config.json       pi@<pi-ip>:~/keyword/
    checkpoints/
scp inference_rpi_awq.py                     pi@<pi-ip>:~/keyword/
```

9.3 Preprocessing Config

`preprocess_config.json` contains all parameters needed to reproduce the mel spectrogram on the Pi without importing the training code:

```
{
    "sample_rate": 16000,
    "n_fft":       512,
    "win_length":  400,
    "hop_length":  160,
    "n_mels":      80,
    "max_frames":  101,
    "keywords":    ["backward", "bed", ...],   // 35 entries
    "cls_token":   [[0.0, 0.0, ..., 0.0]]      // [1, 1, 256]
    FP32 array
}
```

The `cls_token` value is saved because it is a learned parameter that is fed as a secondary input to the model at inference time (PT2E lifts it as a graph input).

10 Inference on Raspberry Pi 3

10.1 Installation

```
# tflite-runtime 2.13 armv7l wheel is available on piwheels
pip install tflite-runtime numpy scipy soundfile sounddevice librosa
```

10.2 Model Loading and Hidden-Input Patching

`AWQSpotter.__init__` loads the model with a retry loop that handles any remaining hidden weight tensors (defensive coding for model files generated by older toolchain versions):

```
for attempt in range(12):
    interp = tflite.Interpreter(model_content=model_bytes,
    num_threads=4)
    interp.allocate_tensors()
    # set mel input and cls_token secondary input
    interp.set_tensor(inp_idx, dummy_mel)
```

```
    for idx, data in extra_inputs:
        interp.set_tensor(idx, data)
    try:
        interp.invoke()
        break           # success
    except RuntimeError as exc:
        # "Input tensor N lacks data" -> patch N out of SubGraph.inputs
        bad = int(re.search(r'Input tensor (\d+) lacks data', str(exc)).group(1))
        model_bytes = _patch_hidden_inputs(model_bytes, [bad])
```

With a correctly generated model (buffer data embedded), the loop succeeds on the first attempt.

10.3 Inference Modes

Mode	Command	Description
Real-time	`-mode realtime`	Captures 1-second chunks from microphone
File	`-mode file -file clip.wav`	Single-file classification
Benchmark	`-mode benchmark`	100-run latency statistics

Table 4: Inference modes on Raspberry Pi 3

10.4 Inference Latency

Typical latency on RPi 3 for a single 1-second clip (INT8, 4 threads, no NEON acceleration in tflite-runtime 2.13):

Statistic	Value
Mel spectrogram compute	∼30 ms (librosa) / ∼80 ms (NumPy fallback)
TFLite invoke	∼120–180 ms
Total (mel + invoke)	∼150–260 ms

Table 5: Approximate latency on Raspberry Pi 3 (armv7l)

11 Adapting the Pipeline to a Different Model

This section summarizes every decision point a practitioner must address when applying this pipeline to a new PyTorch model.

11.1 Prerequisites

1. Model must be **exportable by PT2E**: no dynamic control flow, no data-dependent shapes, no Python-side branching inside `forward`. Replace `if training` with `nn.Dropout` (which is a no-op in eval mode) and explicit mask tensors.

2. All **sample inputs** must match production shapes exactly (batch size, sequence length, feature dimension). PT2E traces with the given shapes; a mismatch causes shape errors at export.
3. The model must be in **eval mode**: `model.eval()` before `ltt.convert()`.

11.2 AWQ Targeting

AWQ is most beneficial for layers that:

- Receive LayerNorm-normalized activations (high inter-channel variance).
- Have large weight matrices (attention projections, FFN layers).

The layers to target are identified by inspecting `named_modules()` for `nn.Linear` instances immediately preceded by `nn.LayerNorm`. The generic version of `compute_awq_scales` should iterate over all such pairs rather than hardcoding block indices.

11.3 Calibration Data

- Use **validation data**, not training data, to avoid calibration overfitting.
- 64 batches $\times$ 32 samples = 2048 examples is typically sufficient. More calibration data rarely improves AWQ significantly.
- The data must pass through the **same preprocessing** as production inputs.

11.4 Quantization Configuration

`ai_edge_quantizer` supports three granularities:

Granularity	API constant	Use case
Per-tensor	`TENSORWISE`	Fastest, lowest accuracy
Per-channel	`CHANNELWISE`	Best accuracy/speed tradeoff for FC layers
Per-block	`BLOCKWISE`	AWQ-style sub-channel (not used here)

For `FULLY_CONNECTED` (all `nn.Linear`), channel-wise is strongly recommended. For `CONV_2D`, per-channel quantization along the output-channel axis is standard.

11.5 Detecting and Handling the mmap Format

`_embed_mmap_buffers` is format-agnostic: it skips any buffer that already has embedded data and is safe to call on any TFLite file. Always call it before deploying to an old `tflite-runtime`. The cost is that the output file is larger, but the data is already in the file anyway (just at different offsets).

11.6 Verifying the Output

Always run `eval_tflite` on the training machine before copying:

```
acc_fp32   = eval_pytorch(model_awq, val_dl, n=30)
acc_tflite = eval_tflite(TFLITE_PATH, val_dl, n=30)
assert acc_fp32 - acc_tflite < 0.02, \
    f"Quantization drop too large: {acc_fp32 - acc_tflite:.4f}"
```

Typical accuracy drop with AWQ + INT8 channel-wise: 0.5–2.0%.

12 Troubleshooting

Error	Cause and fix
`Input tensor N lacks data` on RPi	Two causes: (1) Buffer in mmap format — re-run `quantize_awq.py` with `_embed_mmap_buffers` enabled. (2) Tensor N in `SubGraph.inputs` but not provided — check `_fix_hidden_inputs` log.
`[Warning] Could not locate inputs vector`	`_find_inputs_vec_offset` used the wrong sign for the vtable: `root + i32(root)` instead of `root - i32(root)`. See Equation 21.
`AWQ broke mathematical equivalence`	`apply_awq_scales` modified a layer that `compute_awq_scales` did not target, or the wrong norm/linear pair was matched. Verify with `assert abs(acc_orig - acc_awq) < 0.005`.
`torch.export` dynamic shape error	The model contains a data-dependent branch or a shape that changes with input. Trace with `dynamic_shapes` argument or refactor the model.
`ltt.convert` fails with unsupported op	litert-torch cannot lower a custom operator. Replace with a supported decomposition or use `torch.jit.script` to inline the computation.
Large accuracy drop after INT8 (>3%)	Increase AWQ calibration batches, increase the grid resolution (`AWQ_N_GRID`), or add more layers to the AWQ targeting set.
TFLite file too large for target device	The embedded format doubles the file size. Remove the mmap weight data from the original file by rebuilding the flatbuffer with only the embedded buffers section.

13 Summary of Output Files

File	Size	Contents
`checkpoints/best_model.pt`	∼55 MB	FP32 PyTorch checkpoint
`checkpoints/keyword_spotting_awq.tflite`	∼4.8 MB	INT8 TFLite flatbuffer, classic embedded format, compatible with all `tflite-runtime` versions
`checkpoints/preprocess_config.json`	<1 kB	STFT/mel parameters, keyword list, CLS token value

Table 7: Output artefacts

Copy `keyword_spotting_awq.tflite` and `preprocess_config.json` to the Raspberry Pi.

A FlatBuffers Navigation Reference

Complete self-contained navigation functions for TFLite flatbuffers using only Python's `struct` module.

```python
import struct

def read_model_info(buf: bytes) -> dict:
    """Return key structural addresses from a TFLite flatbuffer."""
    u32 = lambda o: struct.unpack_from("<I", buf, o)[0]
    i32 = lambda o: struct.unpack_from("<i", buf, o)[0]
    u16 = lambda o: struct.unpack_from("<H", buf, o)[0]

    root = u32(0)                       # Model object start
    vtbl = root - i32(root)             # Model vtable  [Eq. 1]
    vtbl_sz = u16(vtbl)

    # Field lookup helper
    def field_off(obj, vtbl, field_n):
        entry = vtbl + 4 + field_n * 2
        if entry + 2 > vtbl + vtbl_sz:
            return 0                    # field not in vtable
        voff = u16(entry)
        return obj + voff if voff else 0   # 0 = field absent

    # Model.subgraphs (field 2)
    sg_ptr = field_off(root, vtbl, 2)
    sg_vec = sg_ptr + u32(sg_ptr) if sg_ptr else 0
    sg_cnt = u32(sg_vec) if sg_vec else 0

    # SubGraph[0]
    sg0 = (sg_vec + 4) + u32(sg_vec + 4) if sg_cnt else 0
```

```
    sg0_vtbl = sg0 - i32(sg0) if sg0 else 0
    sg0_vtbl_sz = u16(sg0_vtbl) if sg0 else 0

    # SubGraph.inputs (field 1)
    inp_ptr = field_off(sg0, sg0_vtbl, 1)
    inp_vec = inp_ptr + u32(inp_ptr) if inp_ptr else 0
    inp_cnt = u32(inp_vec) if inp_vec else 0
    inputs  = [struct.unpack_from("<i", buf, inp_vec + 4 + j * 4)[0]
               for j in range(inp_cnt)]

    # Model.buffers (field 4)
    bufs_ptr = field_off(root, vtbl, 4)
    bufs_vec = bufs_ptr + u32(bufs_ptr) if bufs_ptr else 0
    bufs_cnt = u32(bufs_vec) if bufs_vec else 0

    return {
        "root": root, "vtbl": vtbl, "vtbl_sz": vtbl_sz,
        "sg_count": sg_cnt, "subgraph0": sg0,
        "inputs": inputs,
        "bufs_vec": bufs_vec, "bufs_count": bufs_cnt,
    }
```

B Vtable Field Index Quick Reference

Table	Field name	Field index	Vtable offset
5*Model	version	0	vtbl + 4
	operator_codes	1	vtbl + 6
	subgraphs	2	vtbl + 8
	description	3	vtbl + 10
	buffers	4	vtbl + 12
3*SubGraph	tensors	0	vtbl + 4
	inputs	1	vtbl + 6
	outputs	2	vtbl + 8
3*Buffer	data	0	vtbl + 4
	offset (mmap)	1	vtbl + 6
	size (mmap)	2	vtbl + 8
4*Tensor	shape	0	vtbl + 4
	type	1	vtbl + 6
	buffer	2	vtbl + 8
	name	3	vtbl + 10

Table 8: Vtable entry addresses for commonly accessed TFLite schema fields. Formula: `field_value = u16(vtbl + 4 + field_index * 2)`; if non-zero, the field is at `obj + field_value`.